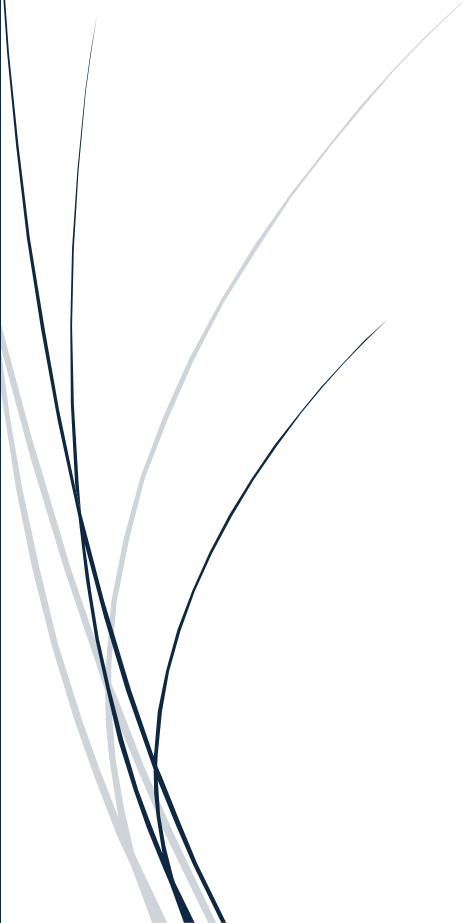




12/06/2026

Smart Asset Management and Resource Allocation Platform

Design Document



ANMOL YADAV (24117016)
B.TECH ELECTRICAL ENGINEERING

HARSHIT AGARWAL (24115071)
B.TECH ELECTRICAL ENGINEERING

AMRUTKAR PRATHAMESH ANIL (24115020)
B.TECH ELECTRICAL ENGINEERING

Table of Contents

Design Document	1
1. Problem Understanding	2
2. System Architecture	2
3. Database Schema	3
4. Entity Relationship Diagram (ERD)	3
5. API & Routing Overview	4
6. Design Decisions	4

1. Problem Understanding

Organizations frequently struggle with managing physical assets (e.g., electronics, tools, vehicles) using manual logs or disjointed spreadsheets. This leads to common operational failures, including:

- **Double-Booking:** Users reserving the same asset for overlapping dates.
- **Inventory Blind Spots:** Lack of real-time visibility into how many units are currently available versus how many are broken or deployed.
- **Poor Accountability:** No clear audit trail of who damaged an item, when it was reported, or who approved a reservation.
- **Friction in Identification:** Difficulty identifying an asset's digital record from the physical item itself.

The Solution: A centralized, web-based platform that automates inventory math, prevents date conflicts during booking, enforces role-based administrative workflows, and bridges the physical-digital gap using dynamically generated QR codes.

2. System Architecture

The application follows a monolithic Model-View-Template (MVT) architectural pattern utilizing the Django framework.

- **Frontend (Template):** HTML5 rendered server-side, styled with Tailwind CSS for a responsive, utility-first UI. Vanilla JavaScript is used for lightweight, dynamic UI toggles (e.g., hiding form fields based on dropdown selections).
- **Backend (View):** Python/Django handles the business logic, request routing, form validation, and authorization gating (Admin vs. Standard User).
- **Database (Model):** Relational database (SQLite) managed through Django's Object-Relational Mapper (ORM).
- **Key Sub-Systems:**
 - **QR Code Generator:** Python `qrcode` library encodes URLs into Base64 image strings dynamically in memory, avoiding heavy file storage.
 - **Notification Engine:** A database-backed alert system that triggers UI state changes based on admin actions.
 - **Conflict Resolution Engine:** Aggregation logic (Sum) that calculates deployed units against requested dates to block overlapping bookings.

3. Database Schema

The system relies on five core relational tables:

Table / Model	Description	Key Fields (Columns)
User	Django's built-in Auth model representing staff and standard users.	id, username, password, is_staff
Asset	The primary physical inventory ledger.	id, name, category, total_quantity, quantity_available, status
Booking	Records of reservation requests and their active states.	id, user_id, asset_id, quantity_requested, start_date, end_date, status
Maintenance Log	Audit trail for broken units and status overrides.	id, asset_id, reported_by_id, issue_description, units_damaged, date_logged
Notification	Real-time alerts dispatched to specific users.	id, user_id, message, is_read, created_at

4. Entity Relationship Diagram (ERD)

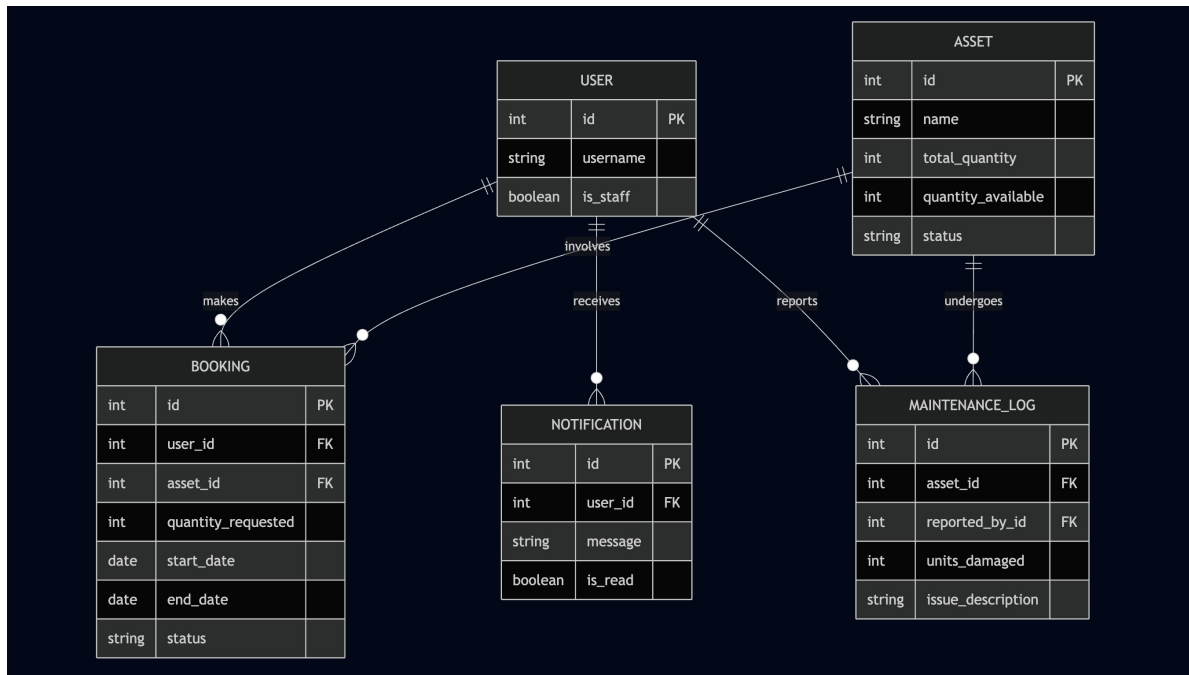


Fig1. Entity Relationship Diagram

5. API & Routing Overview

Because this is a server-rendered application, these endpoints act as internal web routes rather than a headless REST API.

- `GET /` (Dashboard View): Returns aggregate data, active bookings, and the asset catalog.
- `GET /asset/<id>/book/`: Renders the booking form and current availability math for a specific asset.
- `POST /asset/<id>/book/`: Accepts booking parameters (`quantity`, `start_date`, `end_date`), runs conflict validation, and creates a Pending record.
- `GET/POST /asset/<id>/health/`: Protected endpoint (Admin only). Accepts damage logs and modifies physical inventory counts (`quantity_available`).
- `GET /asset/<id>/check/`: The target URL for QR code scans. Redirects to the booking page with a scanned success message.
- `GET /admin-workflow/<booking_id>/<action>/`: Executes state changes (Approve, Reject, Return) on a booking and dispatches a system notification to the requester.

6. Design Decisions

- **Dynamic QR Generation over Static Files**: Instead of generating and saving image files to the server's hard drive whenever an asset is created, the system generates base64-encoded image strings dynamically in Python. This prevents file bloat, ensures the QR code always points to the correct URL, and eliminates broken image links.
- **Two-Tiered Inventory Math**: The Asset model tracks both `total_quantity` and `quantity_available`. This decision allows the system to remember how large the original fleet was, while independently calculating real-time functional limits when processing bookings or logging broken units.
- **Strict State Bypassing for "Out of Stock"**: Rather than forcing admins to manually zero out damaged units when an entire shelf goes missing or is retired, selecting "Out of Stock" dynamically hides the UI requirements via JavaScript and processes a clean ledger override, improving administrative UX.
- **Implicit Notification Routing**: Rather than relying on background workers (like Celery or Redis) which would overcomplicate the deployment architecture, notifications are generated synchronously within the `update_booking_status` view logic. This keeps the codebase lightweight while delivering real-time UI feedback.